

# 17 Working with Spring Data MongoDB

This chapter covers

- Introducing MongoDB
- Examining Spring Data MongoDB
- Accessing a database with MongoRepository
- Accessing a database with MongoTemplate

Document-oriented databases are one type of NoSQL database, where the information is kept as a key/value store. MongoDB is such a database program. Spring Data MongoDB, as part of the larger Spring Data umbrella project, facilitates the interaction of Java programs with MongoDB document databases.

## 17.1 Introducing MongoDB

MongoDB is an open source, document-oriented NoSQL database. MongoDB uses JSON-like documents to store the information, and it uses the concepts of databases, collections, and documents.

- *Database*—A database represents a container for collections. After installing MongoDB, you will generally get a set of databases.
- *Collection*—A collection is similar to a table in the world of relational database management systems (RDBMSs). A collection may contain a set of documents.
- *Document*—A document represents a set of key/value pairs, equivalent to rows in RDBMSs. Documents belonging to the same collection may have different sets of fields. Fields that are common to multiple documents in a collection may contain data of different types—such situations are known as dynamic schemas.

Table 17.1 summarizes the terminology equivalences between relational databases and the NoSQL database MongoDB.

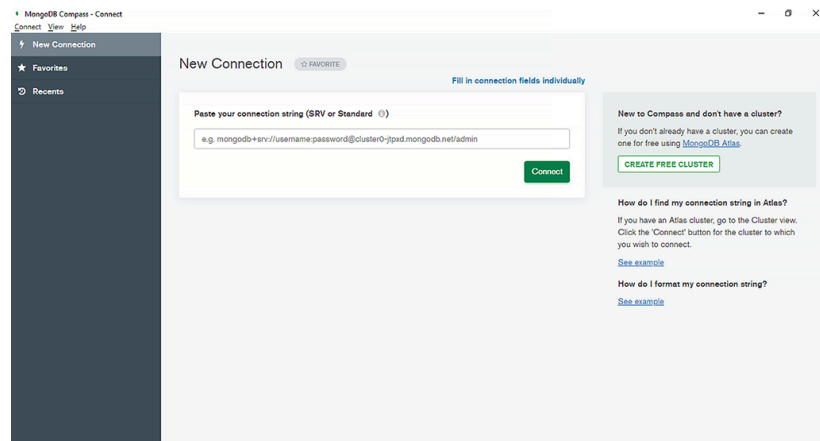
Table 17.1 Compared terminology: RDBMS vs. MongoDB

| Relational databases | MongoDB    |
|----------------------|------------|
| Database             | Database   |
| Table                | Collection |
| Row                  | Document   |
| Column               | Field      |

You can download the MongoDB Community Edition installation kit here: <https://www.mongodb.com/try/download/community>. Installation instructions, depending on your operating system, can be found here:

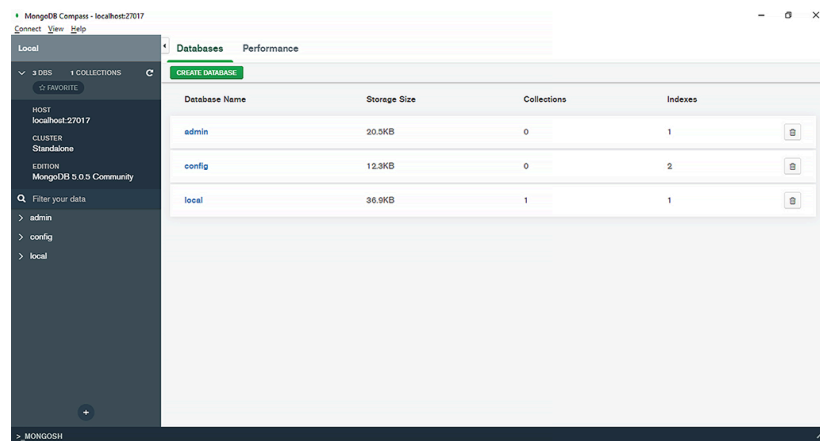
<https://docs.mongodb.com/manual/administration/install-community/>.

After installing MongoDB, you can open the MongoDB Compass program, as in figure 17.1. MongoDB Compass is a GUI for interacting with and querying MongoDB databases.



Listing 17.1 Opening the MongoDB Compass program

Click the Connect button, and you will connect to the local server, as demonstrated in figure 17.2.



Listing 17.2 The connection to the local MongoDB server

Data in a MongoDB collection is represented in JSON format. A typical MongoDB document describing a user of our CaveatEmptor application might look like this:

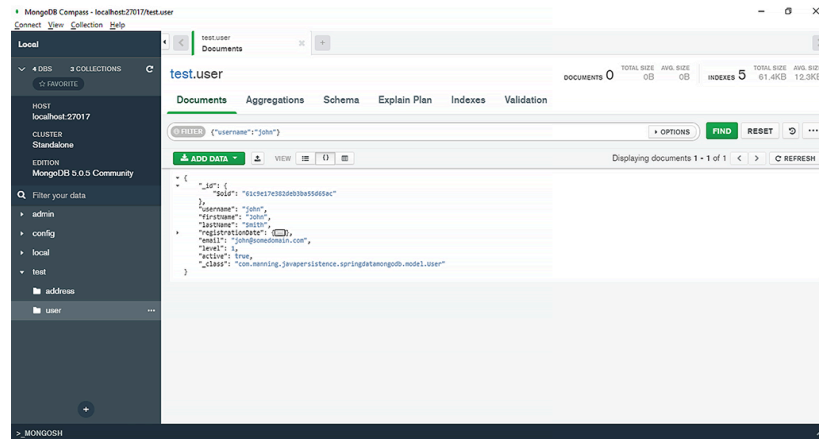
```
{
  "_id": {
    "$oid": "61c9e17e382deb3ba55d65ac"
  },
  "username": "john",
  "firstName": "John",
  "lastName": "Smith",
  "registrationDate": {
    "$date": "2020-04-12T21:00:00.000Z"
  },
  "email": "john@somedomain.com",
  "level": 1,
  "active": true,
```

```

    "_class": "com.manning.javapersistence.springdatamongodb.model.User"
  }
}

```

To select documents fulfilling particular conditions, you can use the Filter edit box in the MongoDB Compass program to insert a query filter parameter. For example, to select documents with the username “john”, you would insert the query filter `{"username": "john"}` and click the Find button, as demonstrated in figure 17.3.



Listing 17.3 Selecting a document from a MongoDB collection

For details about MongoDB CRUD operations you may want to execute, refer to the official documentation:

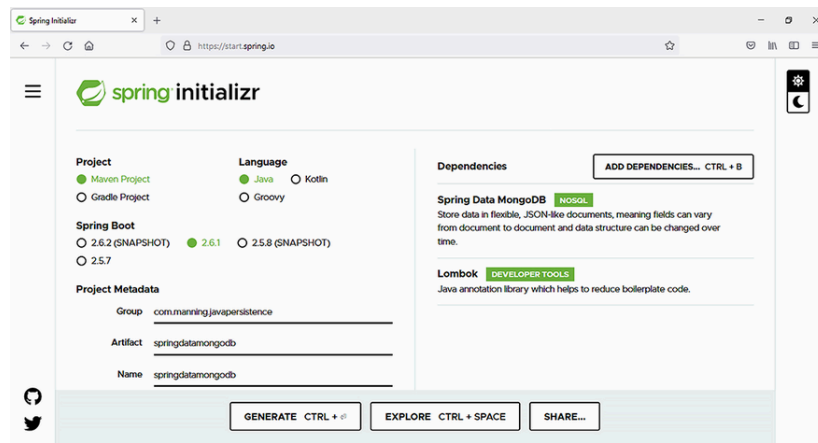
<https://docs.mongodb.com/manual/crud/>.

## 17.2 Introducing Spring Data MongoDB

Spring Data MongoDB is part of the umbrella Spring Data project, and it allows for MongoDB to be used from Java programs, following the Spring Data approach: repositories and custom object-mapping abstractions, annotations, dynamic query creation based on repository method names, integration with other Spring projects, and Spring Boot.

To demonstrate Spring Data MongoDB, we’ll create an application that will manage and persist CaveatEmptor users. We’ll create a Spring Boot application that uses Spring Data MongoDB. To do this, go to the Spring Initializr website (<https://start.spring.io/>) and create a new Spring Boot project (see figure 17.4), having the following characteristics:

- Group: com.manning.javapersistence
- Artifact: springdatamongodb
- Description: Spring Data MongoDB



Listing 17.4 Creating a new Spring Boot project using Spring Data MongoDB

We'll also add the following dependencies:

- Spring Data MongoDB (this will add `spring-boot-starter-data-mongodb` in the Maven `pom.xml` file)
- Lombok (this will add `org.projectlombok,lombok` in the Maven `pom.xml` file)

The `pom.xml` file (listing 17.1) includes the dependencies that we previously added to start the project: the dependency on the Spring Data MongoDB framework and on Lombok. Lombok is a Java library that can be used to automatically create constructors, getters, and setters through annotations, thus reducing the boilerplate code. Lombok has its shortcomings, including these: you will need a plugin for the IDE to understand the annotations and not complain about missing constructors, getters, and setters; and you cannot set breakpoints and debug inside the generated methods (but debugging inside these methods is pretty rare).

Listing 17.1 The `pom.xml` Maven file

Path: `Ch17/springdatamongodb/pom.xml`

```
\1
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-mongodb</artifactId>
</dependency>
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
</dependency>
```

Ⓐ `spring-boot-starter-data-mongodb` is the starter dependency used by Spring Boot to connect to a MongoDB database through Spring Data.

Ⓑ Lombok allows us to reduce boilerplate code and instead rely on automatically generated constructors, getters, and setters.

Our next step is to fill in the Spring Boot `application.properties` file, which can include various properties that will be used by the application. Spring Boot will automatically find and load `application.properties` from the classpath, and the `src/main/resources` folder is added by Maven to the

classpath. The application.properties configuration file is shown in listing 17.2.

Listing 17.2 The application.properties file

```
Path: Ch17/springdatamongodb/src/main/resources/application.properties

logging.level.org.springframework.data.mongodb.core.MongoTemplate=DEBUG  Ⓐ
spring.data.mongodb.auto-index-creation=true                             Ⓑ
```

Ⓐ Queries are logged by the Spring Data MongoDB application at DEBUG level. So, to enable query logging, we have to set the log level to DEBUG.

Ⓑ The creation of indexes is disabled by default in Spring Data MongoDB. Enable it by setting the `spring.data.mongodb.auto-index-creation` property to `true`.

The `User` class will now contain annotations that are specific to Spring Data MongoDB. Table 17.2 examines a few annotations and classes, and then we'll look at them in action while working with the `User` class.

Table 17.2 Spring Data MongoDB annotations and classes

| Spring Data MongoDB annotation/class | Meaning                                                                                                                  |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <code>@Document</code>               | A domain object to be persisted to MongoDB                                                                               |
| <code>@Indexed</code>                | A field indexed by MongoDB                                                                                               |
| <code>@CompoundIndexes</code>        | A container annotation for compound indexes; it defines a collection of multiple <code>@CompoundIndex</code> annotations |
| <code>@CompoundIndex</code>          | Annotates a class to use compound indexes on multiple fields                                                             |
| <code>IndexDirection</code>          | An enum that determines the index direction: <code>ASCENDING</code> (the default) or <code>DESCENDING</code>             |

The `org.springframework.data.mongodb.core.mapping` package includes the `@Document` annotation, while the annotations and the enum related to indexes belong to the `org.springframework.data.mongodb.core.index` package.

For the MongoDB application, we'll also use a series of core Spring Data annotations belonging to the `org.springframework.data.annotation` package, as shown in table 17.3.

Table 17.3 Spring Data core annotations

| Spring Data annotation               | Meaning                                                                                                                     |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <code>@Id</code>                     | Marks the field as an identifier                                                                                            |
| <code>@Transient</code>              | A transient field that will not be persisted and will not be examined by the persistence framework                          |
| <code>@PersistenceConstructor</code> | Marks the constructor to be the primary one used by the persistence framework when retrieving information from the database |

This chapter uses the Lombok library to automatically create constructors, getters, and setters through annotations, thus reducing the boilerplate code. The most important Lombok annotations belonging to the `lombok` package are listed in table 17.4.

Table 17.4 Lombok annotations

| Lombok annotation               | Meaning                                                                            |
|---------------------------------|------------------------------------------------------------------------------------|
| <code>@NoArgsConstructor</code> | Automatically creates a public no arguments constructor for the class it annotates |
| <code>@Getter</code>            | Automatically creates a public getter for the field it annotates                   |
| <code>@Setter</code>            | Automatically creates a public setter for the field it annotates                   |

The `User` class used by the Spring Data MongoDB application is presented in listing 17.3. The `password` field annotated with `@Transient` will not be saved to the MongoDB database—there are many cases where you would like secret information, such as a password, not to be persisted. The constructor annotated with `@PersistenceConstructor` will be used by Spring Data MongoDB when retrieving the information from the database. The `ip` parameter of the constructor is annotated with `@Value("#root.ip ?: '192.168.1.100'")`, which means that if the `ip` value is absent when retrieving the document from the database, it will automatically take this default value.

Listing 17.3 The `User` class

```
Path: Ch17/springdatamongodb/src/main/java/com/manning/javapersistence
➡ /springdatamongodb/model/User.java

@NoArgsConstructor
@Document
@CompoundIndexes({
```

```

        @CompoundIndex(name = "username_email",
            def = "{ 'username' : 1, 'email': 1 }"),
        @CompoundIndex(name = "lastName_firstName",
            def = "{ 'lastName' : 1, 'firstName': 1 }")
    })
    public class User {

        @Id
        @Getter
        private String id;

        @Getter
        @Setter
        @Indexed(direction = IndexDirection.ASCENDING)
        private String username;

        //fields annotated with @Getter and @Setter

        @Getter
        @Setter
        @Transient
        private String password;

        //another constructor

        @PersistenceConstructor
        public User(String username, String firstName, String lastName,
            @Value("#root.ip ?: '192.168.1.100'") String ip) {
            this.username = username;
            this.firstName = firstName;
            this.lastName = lastName;
            this.ip = ip;
        }
    }

```

## 17.3 Using MongoRepository to access a database

The `UserRepository` interface extends `MongoRepository<User, String>`, inheriting the MongoDB-related methods and managing the `User` document, which has IDs of type `String`.

Listing 17.4 The `UserRepository` interface

```

Path: Ch17/springdatamongodb/src/main/java/com/manning/javapersistence
➡ /springdatamongodb/repositories/UserRepository.java

public interface UserRepository extends MongoRepository<User, String> {
}

```

### 17.3.1 Defining query methods with Spring Data MongoDB

We'll add new methods to the `UserRepository` interface so we can query the database for some particular documents and use them in tests.

The purpose of the query methods is to retrieve information from the database. Spring Data MongoDB provides a query builder mechanism similar to the one provided by Spring Data JPA—it will create the behavior of repository methods based on their names. Remember that the

query mechanism removes prefixes and suffixes such as `find...By`, `get...By`, `query...By`, `read...By`, and `count...By` from the name of the method, and it then parses the remainder.

Just like Spring Data JPA, Spring Data MongoDB will look at the return type of the method. If we want to find a `User` and return it in an `Optional` container, the method's return type will be `Optional<User>`.

Listing 17.5 The `UserRepository` interface with new methods

Path: Ch17/springdatamongodb/src/main/java/com/manning/javapersistence  
➡ /springdatamongodb/repositories/UserRepository.java

```
public interface UserRepository extends MongoRepository<User, String> {
    Optional<User> findByUsername(String username);
    List<User> findByLastName(String lastName);
    List<User> findAllByOrderByUsernameAsc();
    List<User> findByRegistrationDateBetween(LocalDate start,
        ➡ LocalDate end);
    List<User> findByUsernameAndEmail(String username, String email);
    List<User> findByUsernameOrEmail(String username, String email);
    List<User> findByUsernameIgnoreCase(String username);
    List<User> findByLevelOrderByUsernameDesc(int level);
    List<User> findByLevelGreaterThanEqual(int level);
    List<User> findByUsernameContaining(String text);
    List<User> findByUsernameLike(String text);
    List<User> findByUsernameStartingWith(String start);
    List<User> findByUsernameEndingWith(String end);
    List<User> findByActive(boolean active);
    List<User> findByRegistrationDateIn(Collection<LocalDate> dates);
    List<User> findByRegistrationDateNotIn(Collection<LocalDate> dates);
}
```

The names of the methods need to follow the rules to determine the resulting query. If the method naming is wrong (for example, the entity property does not match in the query method), we'll get an error when the application context is loaded. Table 17.5 summarizes the usage of the essential keywords in building Spring Data MongoDB query methods and the resulting conditions. For a more comprehensive list, see appendix D.





## conditions

| Keyword          | Example                                                                                           | Condition                                                               |
|------------------|---------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| Is, Equals       | findByUsername(String name)<br>findByUsernameIs(String name)<br>findByUsernameEquals(String name) | {"username"                                                             |
| And              | findByUsernameAndEmail(String username, String email)                                             | {"username"<br>"email": "em                                             |
| Or               | findByUsernameOrEmail (String username, String email)                                             | { "\$or" :<br>name" : "us<br>"email" : "                                |
| LessThan         | findByRegistrationDateLessThan(LocalDate date)                                                    | { "registrat<br>{ "\$lt" : {<br>"date"}}}                               |
| LessThanEqual    | findByRegistrationDateLessThanEqual(LocalDate date)                                               | { "registrat<br>{ "\$lte" :<br>"date"}}}                                |
| GreaterThan      | findByRegistrationDateGreaterThan(LocalDate date)                                                 | { "registrat<br>{ "\$gt" : {<br>"date"}}}                               |
| GreaterThanEqual | findByRegistrationDateGreaterThanEqual(LocalDate date)                                            | { "registrat<br>{ "\$gte" :<br>"date"}}}                                |
| Between          | findByRegistrationDateBetween(LocalDate from, LocalDate to)                                       | "registrati<br>"\$gte" : {<br>"from"}, "\$<br>"\$date" : "              |
| OrderBy          | findByRegistrationDateOrderByUsernameDesc(LocalDate date)                                         | "registrati<br>"\$date" : "                                             |
| Like             | findByUsernameLike(String name)                                                                   | { "username<br>"\$regularExp<br>{ "pattern"<br>"options" :              |
| NotLike          | findByUsernameNotLike(String name)                                                                | { "username<br>"\$not" : {<br>pression" :<br>tern" : "na<br>tions" : "" |
| Before           | findByRegistrationDateBefore(LocalDate date)                                                      | { "registrat<br>{ "\$lt" :<br>"date"}}}                                 |

|                       |                                                          |                                                      |
|-----------------------|----------------------------------------------------------|------------------------------------------------------|
| After                 | <code>findByRegistrationDateAfter(LocalDate date)</code> | <code>{ "registra<br/>{ "\$gt" :<br/>"date"}}</code> |
| Null, IsNull          | <code>findByRegistrationDate(Is)Null()</code>            | <code>{ "registra<br/>null}</code>                   |
| NotNull,<br>IsNotNull | <code>findByRegistrationDate(Is)NotNull()</code>         | <code>{ "registra<br/>{ "\$ne" :vr</code>            |
| Not                   | <code>findByUsernameNot(String name)</code>              | <code>{ "username<br/>: "name"}}</code>              |

As a base class for all future tests, we'll write the `SpringDataJdbcApplicationTests` abstract class (listing 17.6).

The `@SpringBootTest` annotation is added by Spring Boot to the initially created class, and it will tell Spring Boot to search the main configuration class (the `@SpringBootApplication` annotated class, for instance) and create the `ApplicationContext` to be used in the tests. As you'll recall, the `@SpringBootApplication` annotation added by Spring Boot to the class containing the `main` method will enable the Spring Boot auto-configuration mechanism, enable the scan on the package where the application is located, and allow registering extra beans in the context.

Using the `@TestInstance(TestInstance.Lifecycle.PER_CLASS)` annotation, we'll ask JUnit 5 to create a single instance of the test class and reuse it for all test methods. This will allow us to make the `@BeforeAll` and `@AfterAll` annotated methods non-static and to directly use the autowired `UserRepository` instance field inside them. The `@BeforeAll` annotated non-static method is executed once before all tests from any class extending `SpringDataJdbcApplicationTests`, and it saves the list of users created inside the `generateUsers` method to the database. The `@AfterAll` annotated non-static method is executed once after all tests from any class extending `SpringDataJdbcApplicationTests`, and it deletes all users from the database.

Listing 17.6 The `SpringDataJdbcApplicationTests` abstract class

Path: Ch17/springdatamongodb/src/test/java/com/manning/javapersistence  
 ➔ /springdatamongodb/SpringDataMongoDBApplicationTests.java

```

@SpringBootTest
@TestInstance(TestInstance.Lifecycle.PER_CLASS)
abstract class SpringDataJdbcApplicationTests {
    @Autowired                                ❶
    UserRepository userRepository;           ❷

    @BeforeAll
    void beforeAll() {
        userRepository.saveAll(generateUsers());
    }

    private static List<User> generateUsers() {
        List<User> users = new ArrayList<>();

```

```

        User john = new User("john", "John", "Smith");
        john.setRegistrationDate(LocalDate.of(2020, Month.APRIL, 13));
        john.setEmail("john@somedomain.com");
        john.setLevel(1);
        john.setActive(true);
        john.setPassword("password1");

        //create and set a total of 10 users

        users.add(john);
        //add a total of 10 users to the list

        return users;
    }
    @AfterAll
    void afterAll() {
        userRepository.deleteAll();
    }
}

```

Ⓐ Autowire a `UserRepository` instance. This is possible due to the `@SpringBootApplication` annotation, which enables the scan on the package where the application is located and registers the beans in the context.

The next tests will extend this class and use the already populated database. To test the methods that belong now to `UserRepository`, we'll create the `FindUsersTest` class and follow the same recipe for writing tests: call the repository method and verify its results. Recall that, in JUnit 5, it is enough that test classes and methods are package private; they are not required to be public.

Listing 17.7 The `FindUsersTest` class

Path: Ch17/springdatamongodb/src/test/java/com/manning/javapersistence  
 ➡ /springdatamongodb/FindUsersTest.java

```

class FindUsersTest extends SpringDataJdbcApplicationTests{

    @Test
    void testFindAll() {
        List<User> users = userRepository.findAll();
        assertEquals(10, users.size());
    }

    @Test
    void testFindUser() {
        User beth = userRepository.findByUsername("beth").get();
        assertEquals(
            () -> assertNotNull(beth.getId()),
            () -> assertEquals("beth", beth.getUsername())
        );
    }

    @Test
    void testFindAllOrderByUsernameAsc() {

```

```

        List<User> users = userRepository.findAllOrderByUsernameAsc();
        assertAll(() -> assertEquals(10, users.size()),
            () -> assertEquals("beth", users.get(0).getUsername()),
            () -> assertEquals("stephanie", users.get(users.size() -
                1).getUsername()));
    }

    @Test
    void testFindByRegistrationDateBetween() {
        List<User> users = userRepository.findByRegistrationDateBetween(
            LocalDate.of(2020, Month.JULY, 1),
            LocalDate.of(2020, Month.DECEMBER, 31));
        assertEquals(4, users.size());
    }

    //more tests
}

```

### 17.3.2 Limiting query results, sorting, and paging

As in Spring Data JPA and Spring Data JDBC, the `first` and `top` keywords (used equivalently) can limit the results of query methods. The `top` and `first` keywords can be followed by an optional numeric value to indicate the maximum result size to be returned. If this numeric value is missing, the result size will be 1.

`Pageable` is an interface for pagination information, and in practice we use the `PageRequest` class that implements it. This class can specify the page number, the page size, and the sorting criterion.

The use cases we'll solve here will be getting a limited number of users (such as the first user by username or by registration date, or the first users with a given level, sorted by registration date), or a large bunch of users in pages, so we can easily manipulate them.

To limit the query results and do sorting and paging, we'll add the following methods to the `UserRepository` interface.

Listing 17.8 Limiting query results, sorting, and paging in `UserRepository`

```

Path: Ch17/springdatamongodb/src/main/java/com/manning/javapersistence
➡ /springdatamongodb/repositories/UserRepository.java

```

```

Optional<User> findFirstOrderByUsernameAsc();
Optional<User> findTopOrderByRegistrationDateDesc();
Page<User> findAll(Pageable pageable);
List<User> findFirst2ByLevel(int level, Sort sort);
List<User> findByLevel(int level, Sort sort);
List<User> findByActive(boolean active, Pageable pageable);

```

These methods use the query builder mechanism presented in table 17.5, but this time with the purpose of limiting the results of the query, to do sorting and paging. For example, the `Optional<User> findFirstOrderByUsernameAsc()` method will get the first user by its username (the result is an `Optional`, so eventually it may not exist). The `Page<User> findAll(Pageable pageable)` method will get all users

but in pages. We'll write the following tests to verify that these newly added methods work.

Listing 17.9 Testing limiting query results, sorting, and paging

Path: Ch17/springdatamongodb/src/test/java/com/manning/javapersistence  
➡ /springdatamongodb/FindUsersSortingAndPagingTest.java

```
class FindUsersSortingAndPagingTest extends SpringDataJdbcApplicationTests {
    @Test
    void testOrder() {

        User user1 = userRepository.findFirstByOrderByUsernameAsc().get();  Ⓐ
        User user2 =  Ⓐ
            userRepository.findTopByOrderByRegistrationDateDesc().get();  Ⓐ
        Page<User> userPage = userRepository.findAll(PageRequest.of(1, 3));  Ⓑ
        List<User> users = userRepository.findFirst2ByLevel(2,  Ⓒ
            Sort.by("registrationDate"));  Ⓒ

        assertAll(
            () -> assertEquals("beth", user1.getUsername()),
            () -> assertEquals("julius", user2.getUsername()),
            () -> assertEquals(2, users.size()),
            () -> assertEquals(3, userPage.getSize()),
            () -> assertEquals("beth", users.get(0).getUsername()),
            () -> assertEquals("marion", users.get(1).getUsername())
        );

    }

    @Test
    void testFindByLevel() {
        Sort.TypedSort<User> user = Sort.sort(User.class);  Ⓓ

        List<User> users = userRepository.findByLevel(3,  Ⓔ
            user.by(User::getRegistrationDate).descending());  Ⓔ
        assertAll(
            () -> assertEquals(2, users.size()),
            () -> assertEquals("james", users.get(0).getUsername())
        );

    }

    @Test
    void testFindByActive() {
        List<User> users = userRepository.findByActive(true,  Ⓕ
            PageRequest.of(1, 4, Sort.by("registrationDate")));  Ⓕ
        assertAll(
            () -> assertEquals(4, users.size()),
            () -> assertEquals("burk", users.get(0).getUsername())
        );

    }
}
```

Ⓐ The first test will find the first user by ascending order of username and the first user by descending order of registration date.

Ⓑ Find all users, split them into pages, and return page number 1 of size 3 (the page numbering starts with 0).

Ⓒ Find the first 2 users with level 2, ordered by registration date.

- ① The second test will define a sorting criterion on the `User` class. `Sort.TypedSort` extends `Sort` and can use method handles to define properties to sort by.
- ② Find the users of level 3 and sort by registration date, descending.
- ③ The third test will find active users, sorted by registration date, and will split them into pages, and return page number 1 of size 4 (page numbering starts with 0).

### 17.3.3 Streaming results

Query methods returning more than one result can use standard Java interfaces such as `Iterable`, `List`, `Set`. Like Spring Data JPA and Spring Data JDBC, Spring Data MongoDB supports `Streamable`, which can be used as an alternative to `Iterable` or any collection type. It allows us to concatenate `Streamables` and to directly filter and map over the elements.

The use case we'll solve here is getting the results as a stream, without waiting for the whole collection of users or a page of users to be obtained. This way we'll be able to quickly start processing the first results as they flow to us. Unlike collections, a stream can be consumed only once and is immutable.

We'll add the following methods to the `UserRepository` interface.

Listing 17.10 Adding methods that return `Streamable` in `UserRepository` interface

```
Path: Ch17/springdatamongodb/src/main/java/com/manning/javapersistence
➡ /springdatamongodb/repositories/UserRepository.java

Streamable<User> findByEmailContaining(String text);
Streamable<User> findByLevel(int level);
```

We'll write the following test to verify how these newly added methods work to interact with the database and to provide results as streams. A stream is given as a resource of the `try` block to be automatically closed. An alternative is to explicitly call the `close()` method. Otherwise the stream would keep the underlying connection to the database.

Listing 17.11 Testing methods that return `Streamable`

```
Path: Ch17/springdatamongodb/src/test/java/com/manning/javapersistence
➡ /springdatamongodb/QueryResultsTest.java

@Test
void testStreamable() {
    try(Stream<User> result =
        userRepository.findByEmailContaining("someother")
            .and(userRepository.findByLevel(2))
            .stream().distinct()) {
        assertEquals(7, result.count());
    }
}
```

④  
④  
⑤  
⑥

- Ⓐ The test will call the `findByEmailContaining` method, searching for emails containing the "someother" word.
- Ⓑ The test will concatenate the resulting `Streamable` with the `Streamable`, providing the users of level 2.
- Ⓒ It will transform this into a stream and keep the distinct users.
- Ⓓ Check that the resulting stream contains 7 users.

### 17.3.4 The @Query annotation

We can create methods for which a custom query can be specified by using the `@org.springframework.data.mongodb.repository.Query` annotation. With this `@Query` annotation, the method name does not need to follow any naming convention. The custom query will have a MongoDB query filter as an argument, and this query filter can be parameterized.

We'll add new methods to the `UserRepository` interface: they will be annotated with the `@Query` annotation, and their generated behavior will depend on the definitions of these queries. The `value` parameter will indicate the query filter to be executed. The `fields` parameter will indicate the fields to be included or excluded from the result. Table 17.6 summarizes the query operations and the corresponding `@Query` parameters for the most frequent situations. For a comprehensive list, consult the Spring Data MongoDB documentation.

Table 17.6 Query operations and corresponding `@Query` parameters

| Operation                                                      | @Query parameters                                     |
|----------------------------------------------------------------|-------------------------------------------------------|
| Fetch data for a given field                                   | <code>value = { 'field' : ?0 }</code>                 |
| Fetch data for a given regular expression                      | <code>value = { 'lastName' : { \$regex: ?0 } }</code> |
| Fetch data having a field greater than a parameter             | <code>value = { 'field' : { \$gt: ?0 } }</code>       |
| Fetch data having a field greater than or equal to a parameter | <code>value = { 'field' : { \$gte: ?0 } }</code>      |
| Fetch data having a field less than a parameter                | <code>value = { 'field' : { \$lt: ?0 } }</code>       |
| Fetch data having a field less than or equal to a parameter    | <code>value = { 'field' : { \$lte: ?0 } }</code>      |
| Include only one field in the query                            | <code>fields = "{field : 1}"</code>                   |
| Exclude a field from the query                                 | <code>fields = "{field : 0}"</code>                   |



The query filter is the argument of the `@Query` annotation: the `?0` placeholder will reference the first parameter of the method, the `?1` placeholder will reference the second parameter of the method, and so on, as shown in the following listing.

Listing 17.12 Limiting query results, sorting, and paging in `UserRepository`

```
Path: Ch17/springdatamongodb/src/main/java/com/manning/javapersistence
➡ /springdatamongodb/repositories/UserRepository.java
```

```
@Query("{ 'active' : ?0 }")                                ①
List<User> findUsersByActive(boolean active);              ①

@Query("{ 'lastName' : ?0 }")                              ②
List<User> findUsersByLastName(String lastName);          ②

@Query("{ 'lastName' : { $regex: ?0 } }")                  ③
List<User> findUsersByRegexLastName(String regex);        ③

@Query("{ 'level' : { $gte: ?0, $lte: ?1 } }")             ④
List<User> findUsersByLevelBetween(int minLevel, int maxLevel); ④

@Query(value = "{}", fields = "{username : 1}")           ⑤
List<User> findUsernameAndId();                             ⑤

@Query(value = "{}", fields = "{_id : 0}")                ⑥
List<User> findUsersExcludeId();                           ⑥

@Query(value = "{ 'lastName' : { $regex: ?0 } }", fields = "{_id : 0}") ⑦
List<User> findUsersByRegexLastNameExcludeId(String regex); ⑦
```

- ① The `findUsersByActive` method will return the users with a given `active` status.
- ② The `findUsersByLastName` method will return the users with a given `lastName`.
- ③ The `findUsersByRegexLastName` method will return the users with a `lastName` matching the `?0` placeholder, referencing the regular expression that is the first parameter of the method.
- ④ The `findUsersByLevelBetween` method will return the users with a `level` greater than or equal to the `?0` placeholder, referencing the first parameter of the method, and less than or equal to the `?1` placeholder, referencing the second parameter of the method.
- ⑤ The `findUsernameAndId` method will select all users (because the `value` parameter is `{}`) and will return only the `id` and the `username` fields (because the `fields` parameter is `{username : 1}`).
- ⑥ The `findUsersExcludeId` method will select all users (because the `value` parameter is `{}`) and will exclude the `id` from the returned fields (because the `fields` parameter is `{_id : 0}`).
- ⑦ The `findUsersByRegexLastNameExcludeId` method will select users with a `lastName` matching a given regular expression and will exclude the `id` from the returned fields.

Writing tests for these query methods is pretty straightforward and similar to the previous examples. They can be found in the source code for the book.

## 17.4 Query by Example

Query by Example (QBE) was discussed in chapter 4 when we examined Spring Data JPA. It is a querying technique that does not require writing classical queries to include entities and properties. It allows dynamic query creation and consists of three pieces: a probe, an `ExampleMatcher`, and an `Example`.

A probe is a domain object with already set properties. The `ExampleMatcher` provides the rules about matching particular properties. The `Example` puts the probe and the `ExampleMatcher` together and generates the query. Multiple `Examples` can reuse a single `ExampleMatcher`.

As discussed before, these are the most appropriate use cases for QBE:

- When you want to decouple work on the code from the underlying data store API
- When you want to make frequent changes to the internal structure of the domain objects without propagating them to the existing queries
- When you are building a set of static or dynamic constraints to query a repository

QBE has some limitations:

- It supports only starting/ending/containing/regex matching for `String` properties and only exact matching for other types.
- It does not support nested or grouped property constraints, such as 

```
{ "$or": [ { "username": "username" }, { { "lastName": "lastName", "email": "email" } } ] }
```

.

We won't add any more methods to the `UserRepository` interface. Instead we'll write tests to build the probe, the `ExampleMatcher`s, and the `Examples`. The following listing will create a simple probe and a user having with the `lastName` set.

Listing 17.13 Query By Example tests

```
Path: Ch17/springdatamongodb/src/test/java/com/manning/javapersistence
➡ /springdatamongodb/QueryByExampleTest.java
```

```
User probe = new User(null, null, "Smith");           ❶
List<User> result = userRepository.findAll(Example.of(probe)); ❷
assertThat(result).hasSize(2)                             ❸
    .extracting("username").contains("john", "burk");      ❹
```

❶ Initialize a `User` instance and set up a `lastName` for it. This will represent the probe.

❷ Execute the query to find all users matching the probe.

© Verify that the query to find all users matching the probe returned 2 documents, and it contains the `username`s `john` and `burk`.

We can now create an `ExampleMatcher` with the help of the builder pattern. Any `null` reference property will be ignored by the matcher. However, we'll need to explicitly ignore properties that are primitives. If they are not ignored, they will be included in the matcher with their default values, and that would change the generated query.

Listing 17.14 Query By Example with matcher tests

```
Path: Ch17/springdatamongodb/src/test/java/com/manning/javapersistence
➡ /springdatamongodb/QueryByMatcherTest.java
```

```
ExampleMatcher matcher = ExampleMatcher.matching()
                                .withIgnorePaths("level")
                                .withIgnorePaths("active");

User probe = new User();
probe.setLastName("Smith");
List<User> result = userRepository.findAll(Example.of(probe, matcher));

assertThat(result).hasSize(2)
    .extracting("username").contains("john", "burk");
```

Ⓐ Create the `ExampleMatcher` with the help of the builder pattern. We explicitly ignore the `level` and `active` properties, which are primitives. If they were not ignored, they would be included in the matcher with their default values (0 for `level` and `false` for `active`) and would change the generated query.

Ⓑ Create and set a `User` probe.

Ⓒ Execute the query to find all users matching the probe.

Ⓓ Verify that the query to find all users matching the probe returned 2 documents, and it contains the `username`s `john` and `burk`.

## 17.5 Referencing other MongoDB documents

Spring Data MongoDB does not support relationships in the sense of the one-to-one, one-to-many, and many-to-many relationships we examined for relational databases. The framework does not support embedding a document inside another document. However, a document may be referenced from another document using DBRefs. A DBRef will include the collection name and the value of the other document's ID field, and optionally another database name.

To use DBRefs, we'll create another class annotated with `@Document` and another MongoDB repository, and we'll use the `@DBRef` annotation inside the class referencing the newly added document.

The new `Address` class we'll add is shown in the following listing. It is annotated with `@Document`, like any class corresponding to a MongoDB document, and the fields are annotated with the Lombok `@Getter` annotation, indicating the automatic generation of the getters.

Path: Ch17/springdatamongodb2/src/main/java/com/manning/javapersistence  
 ➡ /springdatamongodb/model/Address.java

```
@Document
public class Address {

    @Id
    @Getter
    private String id;

    @Getter
    private String street, zipCode, city, state;

    // . . .

}
```

The new `AddressRepository` interface is shown in the next listing. It extends `MongoRepository<Address, String>`, inheriting the MongoDB-related methods and managing the `Address` document, having IDs of type `String`.

Listing 17.16 The `AddressRepository` interface

Path: Ch17/springdatamongodb2/src/main/java/com/manning/javapersistence  
 ➡ /springdatamongodb/repositories/AddressRepository.java

```
public interface AddressRepository extends
    MongoRepository<Address, String> {
}
```

We'll modify the `User` class to include an `address` field that references the `Address` document. We'll use the `@DBRef` annotation, indicating that this field will be stored using a DBRef. We'll also annotate the field with the `@Field` annotation, which can provide a custom name inside the document.

Listing 17.17 The modified `User` class

Path: Ch17/springdatamongodb2/src/main/java/com/manning/javapersistence  
 ➡ /springdatamongodb/model/User.java

```
@NoArgsConstructor
@Document
@CompoundIndexes({
    @CompoundIndex(name = "username_email",
        def = "{ 'username' : 1, 'email': 1 }"),
    @CompoundIndex(name = "lastName_firstName",
        def = "{ 'lastName' : 1, 'firstName': 1 }")
})
public class User {

    // . . .

    @DBRef
    @Field("address")
```

```

    @Getter
    @Setter
    private Address address;

    // . . .

}

```

A MongoDB document describing a user with an address could look like this:

```

{
  "_id": {
    "$oid": "61cb2fcfff98d570824fef66"
  },
  "username": "john",
  "firstName": "John",
  "lastName": "Smith",
  "registrationDate": {
    "$date": "2020-04-12T21:00:00.000Z"
  },
  "email": "john@somedomain.com",
  "level": 1,
  "active": true,
  "address": {
    "$ref": "address",
    "$id": {
      "$oid": "61cb2fcbff98d570824fef30"
    }
  },
  "_class": "com.manning.javapersistence.springdatamongodb.model.User"
}

```

A MongoDB document describing an address could look like this:

```

{
  "_id": {
    "$oid": "61cb2fcbff98d570824fef30"
  },
  "street": "Flowers Street",
  "zipCode": "1234567",
  "city": "Boston",
  "state": "MA",
  "_class": "com.manning.javapersistence.springdatamongodb.model.Address"
}

```

Note that operations such as save and delete are not cascaded between documents. If we save or delete a document, we have to explicitly save or delete the referenced documents.

Writing tests for working with MongoDB documents that reference other documents is pretty straightforward and similar to the previous examples. They can be found in the source code for the book.

## 17.6 Using MongoTemplate to access a database

`MongoTemplate` is a class that provides access to CRUD operations against MongoDB databases. `MongoTemplate` implements the `MongoOperations` interface. The methods from `MongoOperations` are named similarly to the MongoDB driver `Collection` object methods, to facilitate understanding and use of the API.

### 17.6.1 Configuring access to the database through MongoTemplate

To connect our Spring Boot application to MongoDB, we'll extend the `AbstractMongoClientConfiguration` class. This class provides support for the Java configuration of Spring Data MongoDB. We can connect to MongoDB through an implementation of the `MongoDatabaseFactory` interface and a `MongoTemplate`.

The `AbstractMongoClientConfiguration` class provides two beans that can be used in the Spring Boot application:

```
@Bean
public MongoTemplate mongoTemplate(MongoDatabaseFactory databaseFactory,
                                   MappingMongoConverter converter) {
    return new MongoTemplate(databaseFactory, converter);
}

@Bean
public MongoDatabaseFactory mongoDbFactory() {
    return new SimpleMongoClientDatabaseFactory(this.mongoClient(),
                                                this.getDatabaseName());
}
```

We'll create a `MongoDBConfig` class that extends `AbstractMongoClientConfiguration`, and we'll only override the `getDatabaseName()` method to indicate that our Spring Boot application connects to the `test` database, as shown in the following listing.

Listing 17.18 The `MongoDBConfig` class

```
Path: Ch17/springdatamongodb3/src/main/java/com/manning/javapersistence
➡ /springdatamongodb/configuration/MongoDBConfig.java
```

```
@Configuration
public class MongoDBConfig extends AbstractMongoClientConfiguration {
    @Override
    public String getDatabaseName() {
        return "test";
    }
}
```

### 17.6.2 Executing CRUD operations using MongoTemplate

To insert documents in a database, we can use the `MongoTemplate` `insert` method. The method is overloaded, and the following snippet uses the methods that have an object as an argument, or a collection of objects and their class.

```
Path: Ch17/springdatamongodb3/src/test/java/com/manning/javapersistence
➡ /springdatamongodb/template/SpringDataMongoDBApplicationTests.java
```

```
mongoTemplate.insert(GenerateUsers.address);
mongoTemplate.insert(generateUsers(), User.class);
```

The `save` method has different behavior: if the `id` is already present in the database, it executes an update; otherwise, it executes an insert. The method is also overloaded; the following snippet uses the methods that have an object and the collection name as arguments:

```
Path: Ch17/springdatamongodb3/src/test/java/com/manning/javapersistence
➡ /springdatamongodb/template/SaveUpdateTest.java
```

```
mongoTemplate.save(user, "user");
```

We'll use `org.springframework.data.mongodb.core.query.Query` objects to define criteria, projection, and sorting for retrieving MongoDB documents. Such a `Query` initiated using the default constructor will correspond to all documents in a collection. For example, we can remove all documents from a collection with a snippet like this:

```
Path: Ch17/springdatamongodb3/src/test/java/com/manning/javapersistence
➡ /springdatamongodb/template/SpringDataMongoDBApplicationTests.java
```

```
mongoTemplate.remove(new Query(), User.class);
mongoTemplate.remove(new Query(), Address.class);
```

We can modify `Query` objects using different criteria. Here we'll build a query to look for users having `level 1`:

```
Path: Ch17/springdatamongodb3/src/test/java/com/manning/javapersistence
➡ /springdatamongodb/template/FindAndModifyTest.java
```

```
Query query = new Query();
query.addCriteria(Criteria.where("level").is(1));
```

We can build a query to look for users with a given `username` *and* `email`:

```
Path: Ch17/springdatamongodb3/src/test/java/com/manning/javapersistence
➡ /springdatamongodb/template/FindUsersTest.java
```

```
Query query1 = new Query();
query1.addCriteria(Criteria.where("username").is("mike")
    .andOperator(Criteria.where("email").is("mike@somedomain.com")));
```

We can similarly build a query to look for users with a given `username` *or* `email`:

```
Path: Ch17/springdatamongodb3/src/test/java/com/manning/javapersistence
➡ /springdatamongodb/template/FindUsersTest.java
```

```
Query query2 = new Query(new Criteria().
```

```
orOperator(Criteria.where("username").is("mike"),
Criteria.where("email").is("beth@somedomain.com")));
```

To update a document, we can use an object belonging to the `org.springframework.data.mongodb.core.query.Update` class. Such an object must be set with the new values to replace the old ones. `updateFirst` updates the first document found that matches the given criteria. The following code will find the first document from the `User` class having `level 1` and will update it to `level 2`. We can then get all the remaining users having `level 1` using the `find` method:

```
Path: Ch17/springdatamongodb3/src/test/java/com/manning/javapersistence
➡ /springdatamongodb/template/UpdateFirstTest.java
```

```
Query query = new Query();
query.addCriteria(Criteria.where("level").is(1));

Update update = new Update();
update.set("level", 2);
mongoTemplate.updateFirst(query, update, User.class);

List<User> users = mongoTemplate.find(query, User.class);
```

`updateMulti` updates all the documents found that match the given criteria. The following code will find all the documents from the `User` class having `level 1` and will update them to `level 2`:

```
Path: Ch17/springdatamongodb3/src/test/java/com/manning/javapersistence
➡ /springdatamongodb/template/UpdateMultiTest.java
```

```
Query query = new Query();
query.addCriteria(Criteria.where("level").is(1));

Update update = new Update();
update.set("level", 2);
mongoTemplate.updateMulti(query, update, User.class);
```

The `findAndModify` method is similar to `updateMulti`, but it returns the object before it's modified. In the following snippet, we check that the object returned by the `findAndModify` method still has the old `level` value:

```
Path: Ch17/springdatamongodb3/src/test/java/com/manning/javapersistence
➡ /springdatamongodb/template/FindAndModifyTest.java
```

```
Query query = new Query();
query.addCriteria(Criteria.where("level").is(1));

Update update = new Update();
update.set("level", 2);
User user = mongoTemplate.findAndModify(query, update, User.class);

assertEquals(1, user.getLevel());
```

The `upsert` method will look for a document matching some given criteria. If it finds one, it will update it; otherwise, it will create a new docu-



ment combining the query and update object. The name of the method is a combination of `update` and `insert`, and MongoDB will decide what to do depending on whether the document already exists. The following snippet uses the `getMatchedCount()` method to check that one document matched the query and the `getModifiedCount()` method to check that one document was modified by the update:

```
Path: Ch17/springdatamongodb3/src/test/java/com/manning/javapersistence
    ➤ /springdatamongodb/template/UpsertTest.java
```

```
Query query = new Query();
query.addCriteria(Criteria.where("level").is(1));

Update update = new Update();
update.set("level", 2);
UpdateResult result = mongoTemplate.upsert(query, update, User.class);

assertAll(
    () -> assertEquals(1, result.getMatchedCount()),
    () -> assertEquals(1, result.getModifiedCount())
);
```

You can find comprehensive tests accessing MongoDB through `MongoTemplate` in the source code for the book.

For a comparison of the two approaches, `MongoRepository` and `MongoTemplate`, see table 17.7.

Table 17.7 Comparing `MongoRepository` and `MongoTemplate`

|                              | Strong points                                                                                                                                                                                                                                                                                                                                                    | Weak points                                                                                                                                                                                                                                                                                                      |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>MongoRepository</code> | <ul style="list-style-type: none"> <li>• Follows the approach of Spring Data JPA and Spring Data JDBC repositories.</li> <li>• Allows us to quickly create methods with the query builder mechanism pattern, defining their behavior through their names.</li> </ul>                                                                                             | <ul style="list-style-type: none"> <li>• Provides methods that perform basic CRUD operations working with all the fields of a document.</li> <li>• Updating a document must be executed either in steps (find, modify the relevant fields, save) or using methods annotated with <code>@Query</code>.</li> </ul> |
| <code>MongoTemplate</code>   | <ul style="list-style-type: none"> <li>• Provides atomic operations such as <code>updateFirst</code>, <code>updateMulti</code>, <code>findAndModify</code>, <code>upsert</code>.</li> <li>• Atomic operations favor the work in concurrent applications.</li> <li>• The <code>Update</code> object allows us to choose only the fields to be updated.</li> </ul> | <ul style="list-style-type: none"> <li>• More verbose and more code to write, especially for simple operations.</li> </ul>                                                                                                                                                                                       |

## Summary

- You can create and configure a Spring Data MongoDB project using Spring Boot.
- You can create document classes and use the Spring Data MongoDB annotations to define simple and compound indexes, to mark fields as transient, and to define the persistence constructor.
- You can build custom interfaces extending `MongoRepository` and create custom methods following the query builder mechanism to interact with the MongoDB database.
- You can use the Spring Data MongoDB capabilities for limiting query results, sorting, paging, and streaming the results.
- You can use the `@Query` annotation to define custom queries, and you can work with the Query by Example (QBE) querying technique.

- You can configure an application to use `MongoTemplate` to execute CRUD operations against a MongoDB database.